פייתון למהנדסים 2024, סמסטר א' תשפ"ד 0509-1820

תרגיל בית 2
פונקציות ומילונים

הנחיות כלליות:

- קראו **היטב** את השאלות והקפידו שהתוכניות שלכם פועלות בהתאם לנדרש.
- את התרגיל יש לפתור לבד!
- הקפידו על כללי ההגשה המפורסמים באתר. בפרט, יש להגיש את כל השאלות יחד בקובץ ex2_012345678.py המצורף לתרגיל, לאחר החלפת הספרות 012345678 במספר ת.ז שלכם, כל 9 הספרות כולל ספרת הביקורת.
- מועד אחרון להגשה: כמפורסם באתר.
- יש להקפיד על פלטים מדויקים על פי הדוגמאות.
- אין למחוק את ההערות שמופיעות בשלד.
- **אין לשנות את שמות הפונקציות והמשתנים שמופיעים בקובץ השלד של התרגיל.** עם זאת, אתם רשאים להוסיף משתנים ופונקציות נוספות כראות עינכם.
- **אין להשתמש בספריות חיצוניות (ובפונקציות שלהן).** כלומר, אין להשתמש בפקודת import. **כל פונקציה שלא דורשת פקודה זו מותרת לשימוש** (כלומר, זו פונקציה שהמתרגם (interpreter) מכיר ללא פקודה זו).

הנחיות מיוחדות לתרגיל זה:

- אופן ביצוע התרגיל: בתרגיל זה עליכם לכתוב את תוכן הפונקציות הנתונות ע"י החלפת המילה השמורה pass (המסמלת שורה שלא עושה כלום) במימוש שלכם. (אין חובה להוסיף קריאות שמפעילות את הפונקציות לצורך ההגשה אך ניתן להוסיף כאלו באזור המיועד בקובץ השלד – ראו סעיף הבא).
- בדיקה עצמית: הכנסנו לנוחיותכם את הדוגמאות כבדיקות בסוף קובץ השלד. כל בדיקה שעוברת בהצלחה מדפיסה True. הדוגמאות לא בהכרח מכסות את כל המקרים ולכן הריצו את הפונקציות שלכם על מגוון קלטים שונים כדי לוודא את נכונותן (וודאו כי הפלט נכון וכי התוכנית אינה קורסת). **אין למחוק את הטסטים.**
- ניתן להניח כי הקלט שמקבלות הפונקציות הינו כפי שהוגדר בכל שאלה ואין צורך להתייחס לקלט לא תקין, אלא אם כן נאמר אחרת.
- **בכל השאלות אין להדפיס (print) את הפלט אלא להחזיר (return) אותו!** (עם זאת, ייתכנו מקרים בהם אין דרישה להחזיר ערך מסוים).
- אין להשתמש בספריה חיצונית כגון numpy לצורך פתרון התרגיל.

פונקציות:

שאלה 1:

ממשו את הפונקציה בשם **sum_even_digits(n)** המקבלת מספר שלם (מטיפוס int) וחיובי (גדול מאפס) בשם n . הפונקציה **תחזיר** את סכום הספרות שלו שערכן זוגי.

אין להשתמש בשאלה זו בstr ובשום פונקציה של str.

- אם אין ספרות זוגיות במספר, יוחזר 0.

דוגמאות הרצה:

```
>>> sum_even_digits(5638)
14
```

הסבר: הספרות זוגיות ב-5638 הן 8 ו-6. והסכום שלהן הוא 14.

```
>>> sum_even_digits(137)
0
```

הסבר: כיוון שאין ספרות זוגיות ב-137 מוחזר 0 כברירת מחדל.

דוגמאות הרצה נוספות:

```
>>> sum_even_digits(54984127)
18

>>> sum_even_digits(6)
6
```

שאלה 2:

ממשו את הפונקציה בשם **mult_residuals_of_k(lst, k)** המקבלת רשימה בשם lst ומספר חיובי (גדול מאפס) נוסף k מטיפוס float או int.

הפונקציה **תחזיר** מספר מטיפוס float המייצג את מכפלת השאריות של **המספרים** ברשימה (מטיפוס float או int) שאינם מתחלקים ב-k, תוך **התעלמות** מאלמנטים שאינם מטיפוס float או int.

- במידה ואין מספר שאינו מתחלק ב-k (למשל, כאשר הרשימה ריקה) יוחזר 1.0.

דוגמאות הרצה:

```
>>> mult_residuals_of_k([3, 6, 4, "hello", 11, 9], 3)
2.0
```

הסבר: מבין כל המספרים ברשימה למספר 4 ישנה שארית 1 ולמספר 11 ישנה שארית 2. לשאר המספרים אין שארית. 2*1=2 ולכן התשובה היא 2. הפונקציה התעלמה מהאלמנט "hello".

```
>>> mult_residuals_of_k([45.5, 60, "3", 74, 48], 4)
3.0
```

הסבר: השארית של 45.5 היא 1.5, והשארית של 74 היא 2. הפונקציה התעלמה מהאלמנט "3".

שאלה 3:

ממשו את הפונקציה בשם **count_longest_repetition(s, c)** המקבלת מחרוזת בשם s ומחרוזת נוספת בשם c המכילה אות בודדת באלפבית האנגלי. הפונקציה **תחזיר** את אורך הרצף הארוך ביותר ב-s שמכיל רק את התו הנתון ב-c **כאות גדולה או קטנה**.

- אם האות הנתונה ב-c לא מופיעה בתוך s יש להחזיר 0.

דוגמאות הרצה:

```
>>> s = 'eabbaAAacccaaddd'
>>> count_longest_repetition(s, 'a')
4
```

הסבר: יש שלושה רצפים של האות 'a' בתוך s. ברצף הראשון הוא מופיע פעם אחת (מסומן באדום), ברצף השני הוא מופיע 4 פעמים (מסומן בכחול) - **כאות קטנה וגדולה**, וברצף השלישי הוא מופיע פעמיים (מסומן בירוק). הרצף הארוך הוא 4 ולכן יוחזר 4. משיקולים דומים, *count_longest_repetition(s, 'A')* יחזיר תוצאה זהה.

דוגמאות הרצה נוספות:

```
>>> count_longest_repetition ('CCCccc', 'c')
6
>>> count_longest_repetition ('ab!de', 'z')
0
>>> count_longest_repetition ('', 'z')
0
```

מילונים:

שאלה 4:

בשאלה זו תממשו את שאלה 2 עם שינויים קלים, תוך שימוש במילון באופן הבא:

א. כתבו פונקציה **dict_residuals_of_k(lst, k)**, המקבלת את אותם פרמטרים שהוגדרו בשאלה 2. הפונקציה תחזיר **מילון** הממפה את **המספרים** ברשימה lst לערכי השארית שלהם.

דוגמאות הרצה:

```
>>> dict_residuals_of_k([3, 6, 4, "hello", 11, 9], 3)
{3: 0, 6: 0, 4: 1, 11: 2, 9: 0}
>>> dict_residuals_of_k([45.5, 60, "3", 74, 50], 4)
{45.5: 1.5, 60: 0, 74: 2, 50: 2}
```

ב. כתבו פונקציה **mult_residuals_of_k_with_dict(lst, k)**, המקבלת את אותם פרמטרים שהוגדרו בשאלה 2. הפונקציה תשתמש בפונקציה שהגדרתם בסעיף הקודם (4 א) על מנת להחזיר מכפלת השאריות של **המספרים הזוגיים**.

שימו לב: בדומה לשאלה 2, גם בסעיף זה יש להכפיל רק את שאריות המספרים שאינם מתחלקים בk.

דוגמאות הרצה:

```
>>> mult_residuals_of_k_with_dict([3, 6, 4, "hello", 11, 9], 3)
1
>>> mult_residuals_of_k_with_dict([45.5, 60, "2", 74, 50], 4)
4
```

שאלה 5:

נרחיב את שאלה 3, תוך שימוש במילון באופן הבא:

א. כתבו פונקציה **dict_longest_repetition(s)**, המקבלת את מחרוזת s בדומה לשאלה 3. הפונקציה תחזיר **מילון** הממפה **תוים (בודדים)** המופיעים בs לרצפים הארוכים ביותר שלהם.

הבהרות:

- גם בשאלה זו לא נבדיל בין אותיות קטנות לגדולות בs.
- כל מפתח יהיה אות בודדת קטנה המוכלת במחרוזת s.
- המילון לא יכיל אותיות אשר אינן מופיעות בs.
- מותר (אך לא חובה) להיעזר בפונקציה שכתבתם בשאלה 3.

דוגמאות הרצה:

```
>>> dict_longest_repetition('eabbAaaacccaaddd')
```

```
{'e': 1, 'a': 4, 'b': 2, 'c': 3, 'd': 3}
>>> dict_longest_repetition('cccccc')
{'c': 6}
>>> dict_longest_repetition('abcdee')
{'a': 1, 'b': 1, 'c': 1, 'd': 1, 'e': 2}
```

ב. כתבו פונקציה **count_longest_repetition_with_dict(s,c)** המקבלת את אותם פרמטרים שהוגדרו בשאלה 3. הפונקציה תשתמש בפונקציה שהגדרתם בסעיף הקודם (5 א) על מנת להחזיר את אורך הרצף הארוך ביותר של c.

אין להשתמש בלולאות for בסעיף זה.

<u>דוגמאות הרצה:</u>

```
>>> count_longest_repetition_with_dict('eabbAaaacccaaddd', 'a')
4
>>> count_longest_repetition_with_dict('cccccc','c')
6
>>> count_longest_repetition_with_dict('abcdee', 'z')
0
```

ג. כתבו פונקציה **find_longest_repetitions_with_dict(d)** המקבלת את המילון שחזר כפלט בסעיף 5 א. הפונקציה תחזיר את התו המופיע ברצף הארוך ביותר של s. ניתן להניח כי קיים תו אחד כזה בלבד.

```
>>> find_longest_repetitions_with_dict({'e': 1, 'a': 4, 'b': 2,
'c': 3, 'd': 3})
'a'
>>> find_longest_repetitions_with_dict({'c': 6})
'c'
>>> find_longest_repetitions_with_dict({'a': 1, 'b': 1, 'c': 1,
'd': 1, 'e': 2})
'e'
```

<u>שאלה 6</u>

א. ממשו פונקציה בשם **find_substring_locations(s, k)** המקבלת מחרוזת s ואורך k של תת-מחרוזת כמספר שלם **ומחזירה** את המילון הבא:

- המפתחות הם כל תתי המחרוזות של המחרוזת s באורך k (אוסף רציף של תווים מתוך s).
- הערך המתאים לכל מפתח הוא רשימה של כל האינדקסים בהם הוא מופיע ב-s (כל מיקום מצוין על ידי אינדקס התו הראשון של התת-מחרוזת). למשל "ge" מתוך "drge" יופיע במקום 2.

דוגמאות הרצה:

```
>>> find_substring_locations('TTAATTAGGGGCGC', 2)
{'TT': [0, 4], 'TA': [1, 5], 'AA': [2], 'AT': [3], 'AG': [6],
'GG': [7, 8, 9], 'GC': [10, 12], 'CG': [11]}
>>> find_substring_locations('TTAATTAGGGGCGC', 3)
{'TTA': [0, 4], 'TAA': [1], 'AAT': [2], 'ATT': [3], 'TAG': [5
], 'AGG': [6], 'GGG': [7, 8], 'GGC': [9], 'GCG': [10], 'CGC':
[11]}
>>> find_substring_locations('Hello World', 3)
{'Hel': [0], 'ell': [1], 'llo': [2], 'lo ': [3], 'o W': [4],
' Wo': [5], 'Wor': [6], 'orl': [7], 'rld': [8]}
```

כפי שניתן לראות, הפונקציה מחלקת את המחרוזת לתת-מחרוזות באורך k, בצורה רציפה, ומוצאת את כל מיקומי ההתחלה של כל תת-מחרוזת במחרוזת s.

הפונקציה צריכה לדעת להתמודד עם k בגודל: $1 \leq k \leq len(s)$ (כלומר ניתן להניח שזה מתקיים).

הערה:

האינדקסים ברשימות אינם שליליים (כלומר, האינדקסים הם גדולים או שווים ל-0).

ב. ממשו פונקציה בשם **count_substring_locations(substr_dict)** המקבלת כקלט את המילון substr_dict שהוא פלט הפונקציה מהסעיף הקודם. הפונקציה **תשנה** את המילון substr_dict כך שלכל מפתח ימופה **מספר** תתי המחרוזות שנמצאו במחרוזת s מהסעיף הקודם (6 א).

- **שימו לב, הפונקציה אינה מבצעת** return **(כלומר, אסור לפונקציה להחזיר את המילון).**

דוגמאות הרצה:

```
>>> substr_dict=find_substring_locations('TTAATTAGGGGCGC', 2)
>>> count_substring_locations(substr_dict)
>>> substr_dict
{'TT': 2, 'TA': 2, 'AA': 1, 'AT': 1, 'AG': 1, 'GG': 3, 'GC':
2, 'CG': 1}
>>> substr_dict=find_substring_locations('TTAATTAGGGGCGC', 3)
>>> count_substring_locations(substr_dict)
>>> substr_dict
{'TTA': 2, 'TAA': 1, 'AAT': 1, 'ATT': 1, 'TAG': 1, 'AGG': 1,
'GGG': 2, 'GGC': 1, 'GCG': 1, 'CGC': 1}
>>> substr_dict=find_substring_locations('Hello World', 3)
>>> count_substring_locations(substr_dict)
>>> substr_dict
```

```
{'Hel': 1, 'ell': 1, 'llo': 1, 'lo ': 1, 'o W': 1, ' Wo': 1,
'Wor': 1, 'orl': 1, 'rld': 1}
```

מטריצות:

בשאלות הבאות נעבוד עם רשימות דו ממדיות של מספרים ממשיים לטובת ייצוג מטריצות. מטריצה שממדיה n על m (כלומר, יש לה n שורות שאורך כל אחת מהן m) תיוצג על ידי רשימה של n רשימות באורך m, שכל אחת מהן מייצגת שורה אחת.

לדוגמה, מטריצה בעלת 3 שורות ו-2 עמודות שערכיה הם:

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$$

תיוצג על ידי הרשימה:

[[1, 2], [3, 4], [5, 6]]

דוגמא נוספת, מטריצה בעלת שורה אחת שערכיה הם:

$$\begin{pmatrix} 5 & 8 & 3 \end{pmatrix}$$

תיוצג על ידי הרשימה:

[[5, 8, 3]]

שאלה 7:

ממשו הפונקציה בשם **mult_mat_by_scalar(mat, alpha)** המקבלת מטריצה תקנית mat ומספר alpha (מטיפוס int). הפונקציה **תחזיר** מטריצה חדשה בעלת ממדים זהים לאלו של mat כאשר כל אחד מאברי המטריצה המוחזרת היא מכפלת האיבר המתאים ב-mat ב-alpha.

- **יש להחזיר מטריצה חדשה, מבלי לשנות את mat.**
- ניתן להניח שמטריצת הקלט לא ריקה.

דוגמאות הרצה:

```
>>> mat1 = [[2, 5], [6, 9]]
>>> mat2 = mult_mat_by_scalar(mat1, 2)
>>> print(mat1)
[[2, 5], [6, 9]]
>>> print(mat2)
[[4, 10], [12, 18]]
>>> mult_mat_by_scalar([[10,15], [-3,6]], -5)
[[-50, -75], [15, -30]]
```

שאלה 8:

ממשו את הפונקציה בשם **mat_transpose(mat)** המקבלת מטריצה <u>תקנית</u> mat ומחזירה מטריצה חדשה שהיא המטריצה המשוחלפת של mat. שחלוף מטריצה היא פעולת ההחלפה בין השורות והעמודות של מטריצה נתונה. הפעולה מקבלת מטריצה בת n שורות ו-m עמודות, ומחזירה מטריצה בת m שורות ו-n עמודות, שבמקום ה-(i, j) שלה נמצא האיבר ה-(j, i) של המטריצה המקורית.

הנה דוגמה למטריצה והמטריצה המשוחלפת שלה:

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}^T \rightarrow \begin{pmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{pmatrix}$$

- **יש <u>להחזיר</u> מטריצה חדשה, מבלי לשנות את mat.**
- ניתן להניח שמטריצת הקלט לא ריקה.

<u>דוגמאות הרצה:</u>

```
>>> mat = [[1,2],[3,4],[5,6]]
>>> mat_T = mat_transpose(mat)
>>> print(mat)
[[1, 2], [3, 4], [5, 6]]
>>> print(mat_T)
[[1, 3, 5], [2, 4, 6]]
>>> mat2 = [[0, 1, 2], [10, 11, 12], [20, 21, 22]]
>>> mat2_T = mat_transpose(mat2)
>>> print(mat2_T)
[[0, 10, 20], [1, 11, 21], [2, 12, 22]]
```

<u>שאלה 9:</u>

בתרגול המילונים ראינו ייצוג של **sparse matrix** באמצעות מילון. בייצוג כזה, עבור כל תא במטריצה שאינו אפס, יאוחסן במילון מפתח מסוג tuple המייצג את קורדינטות התא, והערך ייצג את ערכו של התא במטריצה.

א. ממשו שחלוף מטריצה כפי שעשיתם בשאלה 8, אך הפעם השתמשו בייצוג sparse matrix כקלט וכפלט.

<u>דוגמת הרצה:</u>

```
>>> mat_transpose_sparse({(2,9):2, (3,4):4})
{(9, 2): 2, (4, 3): 4}
```

ב. ממשו את הפונקציה **diff_sparse_matrices(lst)** אשר מקבלת רשימה lst של מילונים המייצגים sparse matrices **ומחזירה** מילון המייצג את מטריצת ההפרש.

<u>דוגמאות הרצה:</u>

```
>>> diff_sparse_matrices([{(1, 3): 2, (2, 7): 1}, {(1, 3): 6}])
{(1, 3): -4, (2, 7): 1}
>>> diff_sparse_matrices([{(1, 3): 2, (2, 7): 1}, {(1, 3): 2}])
{(2, 7): 1}
>>> diff_sparse_matrices([{(1, 3): 2, (2, 7): 1}, {(1, 3): 6, (9
,10): 7}, {(2,7): 0.5, (4,2): 10}])
{(1, 3): -4, (2, 7): 0.5, (9, 10): -7, (4,2): -10}
```

במקרה השני – ערך מטריצת ההפרש במיקום **(1,3)** הוא 0. לכן, ערך זה הוסר מהמילון.

הערות:

- הפונקציה אמורה לקבל רק רשימה המכילה שתיים או יותר מטריצות מהתצורה הנ"ל.
- אורכן לא חייב להיות זהה (תזכורת: ערך שלא מופיע במילון שווה ל-0).
- במידה ויש יותר משתי מטריצות ברשימה, יש להחסיר גם את השלישית והלאה מהראשונה:

Final = A-B-C

הנחיות:

- ניתן להניח שמימדי המטריצות זהים (כלומר, מספר השורות ומספר העמודות בכל מטריצה זהה למטריצות האחרות).
- ניתן להניח שהפונקציה מקבלת רשימה של לפחות שתי מטריצות מאותו מימד.

בהצלחה!